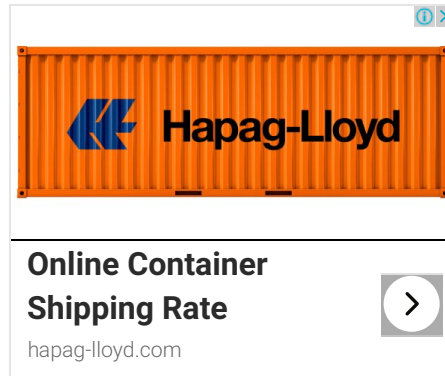


Node.js Basics

Node.js supports JavaScript. So, JavaScript syntax on Node.js is similar to the browser's JavaScript syntax.

Visit [JavaScript](#) section to learn about JavaScript syntax in detail.



Primitive Types

Node.js includes following primitive types:

- > String
- > Number
- > Boolean
- > Undefined
- > Null
- > RegExp

Everything else is an object in Node.js.



Loose Typing

JavaScript in Node.js supports loose typing like the browser's JavaScript. Use `var` keyword to declare a variable of any type.



BIRKENSTOCK

Object Literal

Object literal syntax is same as browser's JavaScript.

Example: Object

 Copy

```
var obj = {  
  authorName: 'Ryan Dahl',  
  language: 'Node.js'  
}
```

Functions

Functions are first class citizens in Node's JavaScript, similar to the browser's JavaScript. A function can have attributes and properties also. It can be treated like a class in JavaScript.

Example: Function

 Copy

```
function Display(x) {  
  console.log(x);  
}  
  
Display(100);
```

Buffer

Node.js includes an additional data type called Buffer (not available in browser's JavaScript). Buffer is mainly used to store binary data, while reading from a file or receiving packets over the network.

process object

Each Node.js script runs in a process. It includes **process** object to get all the information about the current process of Node.js application.

The following example shows how to get process information in REPL using **process** object.

```
> process.execPath
'C:\\Program Files\\nodejs\\node.exe'
> process.pid
1652
> process.cwd()
'C:\\'
```

Defaults to local

Node's JavaScript is different from browser's JavaScript when it comes to global scope. In the browser's JavaScript, variables declared without `var` keyword become global. In Node.js, everything becomes local by default.

Access Global Scope

In a browser, global scope is the window object. In Node.js, **global** object represents the global scope.

To add something in global scope, you need to export it using `export` or `module.export`. The same way, import modules/object using `require()` function to access it from the global scope.

For example, to export an object in Node.js, use `exports.name = object`.

Example:

 Copy

```
exports.log = {
  console: function(msg) {
    console.log(msg);
  },
  file: function(msg) {
    // log to file here
  }
}
```

Now, you can import `log` object using `require()` function and use it anywhere in your Node.js project.

Learn about modules in detail in the next section.

Want to check how much you know Node.js?

[Start Node.js Test](#)



Share



Tweet



Share



Whatsapp

[< Previous](#)

[Next >](#)

.NET Tutorials

C#